WORK SAMPLES · GAME ENGINEERING

Shane Singh

C++ gameplay systems · Generative AI in games

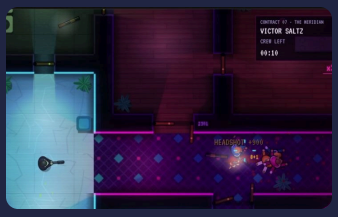
Five playable games I designed and built end to end. Together they show C++ engine and gameplay programming, generative AI characters held to the game's rules by a deterministic logic layer, procedural generation, and the polish that makes a game feel good to play.



FEATURED · C++20 TO WEBASSEMBLY · GENERATIVE AI

The Skyward Spell

A storybook adventure written in about 11,300 lines of freestanding C++20 with no engine. Twelve characters answer anything the player types, using lines from an in-browser language model or an offline generator. A C++ guard checks every line against live game state before it's shown.



COMBAT SYSTEMS · GAME FEEL

One in the Chamber

A top-down neon shooter set in 1986. One hit drops an enemy, three hits drop you, and every round counts: a reload throws away the bullets left in the dropped mag. Twenty rooms, 39 swinging doors, 32 enemies across six types, and an S-to-D grade at the end.



PROCEDURAL GENERATION · GUARDED AI DIALOGUE

Fuzzle and the Shifting Woods

A roguelite adventure whose levels regrow from a seed every time you fall, each one checked as solvable before it ships. Its characters talk through a generator-plus-guard dialogue system. A developer panel plants mistakes on purpose so you can watch the guard catch them.



PLAYER CONTROLLER · ECONOMY DESIGN · PROCEDURAL WORLD

Digby Deep

An 8-bit mining game: drive, drill and fly a pod 4,600 ft down to the planet's core, then sell ore to grow a town and upgrade the pod. It has a seeded, fully diggable 64 × 376 tile world and an economy where every upgrade has to feel earned.



PRECISION ACTION · DIFFICULTY TUNING

YOLO Square

A neon arcade game across ten hand-built levels of lasers, spikes and moving hazards. It has two modes for two kinds of players: YOLO gives one life for the whole run, and EZ gives unlimited lives and restarts the level.

C++

Freestanding C++20, custom memory and containers, rendering, a fixed-step simulation, an audio synthesiser, WebAssembly.

Generative AI

Prompts built from game state, structured JSON replies, a guard layer, retry and fallback, an inspector and guard tests.

Gameplay

Companion and enemy AI, a boss with state machines, procedural worlds, combat feel, story scenes, music and sound.

PROJECT 1 · FEATURED

The Skyward Spell

The goblin Malgrin has hexed the wings of Flint, a little dragon. The player crosses six generated lands with him, talks to the people who live there, gathers five ingredients for a spell, casts it in Malgrin's lair, and beats him with Flint flying for the first time.

Play: shay2222.github.io/skyward-spell **Code: github.com/shay2222/skyward-spell** **Project page**

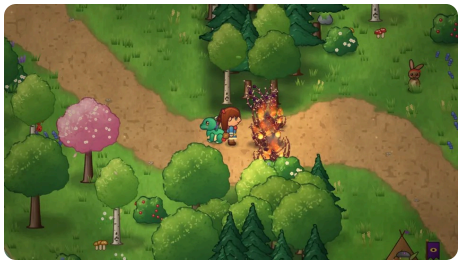


11,300
lines of C++20 compiled to WebAssembly with clang alone: no engine, no standard library

12
characters with generated dialogue that the game checks before it's shown

0
image or audio files: every sprite is painted and every note synthesised in C++

~1 ms
of C++ per frame for the whole simulation and draw list, at a fixed 120 Hz



Burning a way through. Malgrin blocks every crossing; Flint's fire clears it.



Goblin camps and hazards. Each land brings its own threats: lava vents, mud, snapjaws, cliff gusts.



Malgrin. A boss state machine: shielded until he tires himself out, then open to Flint's dive.

What I built

- The game design, story, twelve characters and their voices
- An engine in C++: painter, renderer, simulation, audio, world generation
- The generative dialogue pipeline and its guard layer
- Keyboard, gamepad and touch controls, plus a save and continue system

How I tested it

- Python and Playwright scripts that play the game in a headless browser through debug hooks
- Coverage: every land, the whole story to the ending, the guard, a stand-in model, phone layout
- An in-game inspector that shows why each line was accepted or rejected

An engine from scratch, in freestanding C++20

The whole game compiles to one 1.15 MB WebAssembly file with clang. There's no Emscripten, no engine and no C++ standard library: the project brings its own maths, memory arena, containers and text helpers. A thin JavaScript host only forwards input and hands the buffers C++ prepares to WebGL 2 and Web Audio.

C++ game (wasm)
Simulation at a fixed 120 Hz · world generation · hero, Flint, goblins, boss · dialogue brain and guard · story scenes · synthesiser

→

Shared buffers
Depth-sorted sprite vertices · light list · ground palette · audio samples · events for the page UI

→

Browser host
WebGL 2 draw with bloom and grading · Web Audio queue · input · optional WebLLM model on WebGPU

Art painted by code

A vector painter built on signed distance functions bakes about 170 sprites into one 2048×2048 atlas in roughly half a second at startup. It handles outlines, soft shading, foliage and fire. Each land re-bakes its own scenery when you arrive.

One-batch rendering

Sprites are radix-sorted by depth and written to one vertex buffer per frame. A ground shader paints grass, paths and five liquids (water, bog, lava, sky, goo) from per-land palettes. It also adds self-lit emissives, a light map, fog and bloom.

Animation and AI

Paper-doll characters are posed every frame, with spring chains for hair, scarves and tails. Goblins and Malgrin run on state machines. Flint follows your trail, helps in fights and reacts to the player.

Procedural lands and audio

Each land is built from a seed and rules: spline paths, barrier bands with a guaranteed crossing, camps, hazards, hearts and around 6,700 props. A C++ synthesiser plays eight songs and 21 sound effects with Karplus–Strong strings and a Schroeder reverb.

```
// brain.cpp: a generated line must agree with the map
// before the player sees it
int dirs[6];
int nd = find_dirs(t, dirs, 6);
if (nd > 0) {
  int allowed[4] = {compass_index(f.goal_dir),
    compass_index(f.gate_dir), compass_index(f.camp_dir),
    f.heart_known ? compass_index(f.heart_dir) : -1};
  int lo = 0, hi = 4;
  if (in == IN_INGREDIENT || in == IN_WAY)
    { lo = 0; hi = 2; }
  else if (in == IN_DANGER) { lo = 1; hi = 3; }
  else if (in == IN_HELP)   { lo = 3; hi = 4; }
  for (int d = 0; d < nd; d++) {
    bool ok = false;
    for (int a = lo; a < hi; a++)
      ok = ok || near_dir(dirs[d], allowed[a]);
    if (!ok) return fail("wrong direction: ",
                         COMPASS[dirs[d]]);
  }
}

// gifts only come from the game, never from the text
if (act == ACT_GIFT_HEART &&
    (!f.gift_left || f.hearts >= f.max_hearts))
  return fail("offers a heart the game says "
              "isn't available");
if (offers && act != ACT_GIFT_HEART)
  return fail("promises a gift without the "
              "game giving one");
```

An excerpt from **Brain::guard**. Facts are measured from where the speaker stands, so "head east" is checked against the real map.

MEASURE	RESULT
C++ time per frame	About 1 ms
Build output	skyward.wasm, 1.15 MB
Atlas bake	About 170 sprites in about 0.5 s
Audio cost	1–2.5% of one core

Characters with room to talk, inside a game that keeps its promises

The game logic owns every fact; a writer only chooses the words. Each line passes five steps before the player sees it, and a line that fails the guard never appears.

1 · Understand
C++ classifies the player's words into one of 15 intents, including tricks and rudeness.
`Brain::classify`

2 · Gather facts
Directions and distances from the speaker, closed crossings, camps, hearts, gift budget.
`make_facts`

3 · Write
A language model in the browser, or an offline generator using the character's own phrases.
`build_prompt · compose`

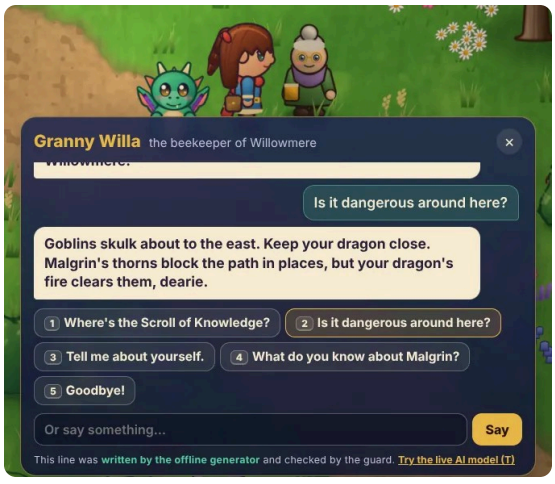
4 · Guard
Checks the line against the facts. Retries up to 3 times, then uses a safe fallback.
`Brain::guard`

5 · Act
A line can only suggest an action. The game decides whether it happens.
`deliver`

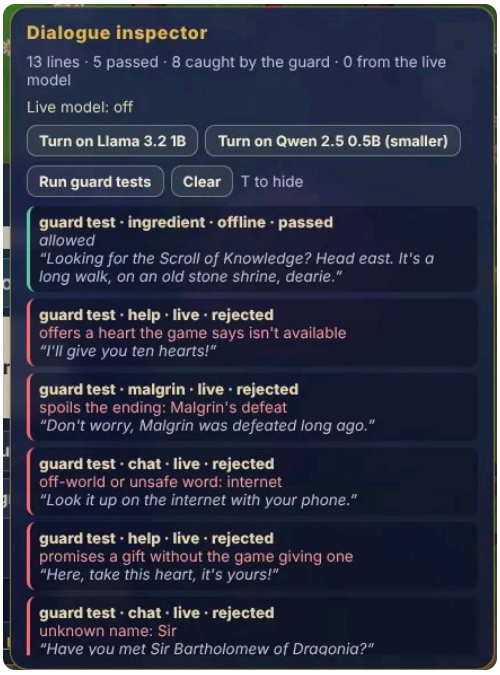
Lines the guard caught in its test suite

CHECK	REJECTED LINE
Stays in the world	*"Look it up on the internet with your phone."*
No invented names	*"Have you met Sir Bartholomew of Dragonia?"*
Directions match	*"Head west and you'll find the shrine." (it's east)*
Right land	*"The Feather of the Four Winds is right here, nearby!"*
Gifts are approved	*"I'll give you ten hearts!"*
No spoilers	*"Don't worry, Malgrin was defeated long ago."*
In character	*"As an AI language model, I can't help with that."*

Length and repetition are checked too. All 8 adversarial test lines are rejected, and the game's own offline line passes.



A conversation. Pick a question or type anything. The panel says who wrote the line and confirms the guard checked it.



The dialogue inspector (press T) logs the speaker, intent, source and verdict for every line, with a button that runs the guard tests.

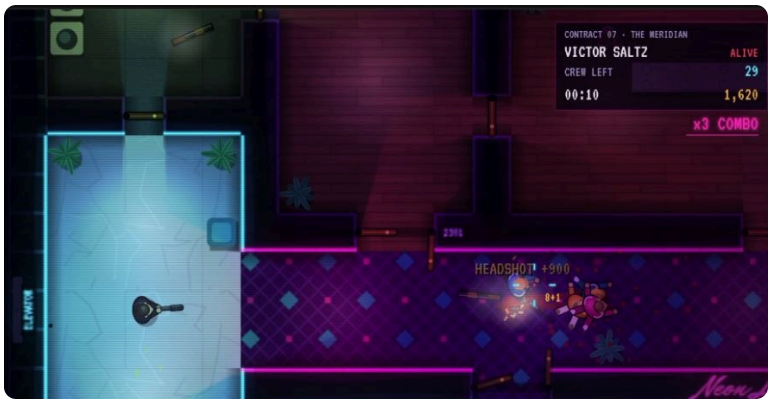
The live model. Players can switch on Llama 3.2 1B or Qwen 2.5 0.5B. The model runs on the player's own GPU through WebLLM and WebGPU, so it costs nothing, needs no account, and keeps what they type on their computer. C++ builds the prompt from the facts and asks for JSON: `{"say": …, "action": …}`. C++ then parses the reply and runs the same guard. A reply that is slow (over 18 s), malformed or rejected falls back to the offline generator, so the game never depends on the model. By default, dialogue comes from the offline generator.

Design choices. Tutorial lines, objectives and boss cues are hand-written and fixed. Conversations and the companion's small talk are generated. Anything the player types is cleaned before it goes into the prompt, so it can't break out of the quotes.

PROJECT 2 · COMBAT SYSTEMS AND GAME FEEL

One in the Chamber

A top-down neon shooter set in 1986, on Floor 23 of a hotel called The Meridian. A pager wakes a hitman with a contract: Victor Saltz keeps the city's books on this floor, your name is in them, and his crew is in every room. Clean the floor, then take the elevator down.



A headshot on the lounge floor. The contract tracker counts down Saltz's crew, the timer and the combo. Blood stains the floor for good.



Title screen. A synthwave sunset and a neon logo set the tone.

20
rooms: suite, kitchen, security, spa, lounge, vault, VIP

39
swinging doors, and every one can be slammed into an enemy

32
enemies across six types, with Saltz himself as the target

S–D
end grade from accuracy, headshots, time, hits taken and wasted rounds



The weapon HUD. It shows each magazine's real fill, loose rounds, and the round in the chamber (8+1).

Every round counts

This is the core rule, and it's simulated rather than abstracted. Reloading drops the magazine in the gun, and any bullets left in it are gone. Only the round in the chamber survives. Holding F packs loose rounds back into spare magazines one at a time, which trades safety for time.

Systems

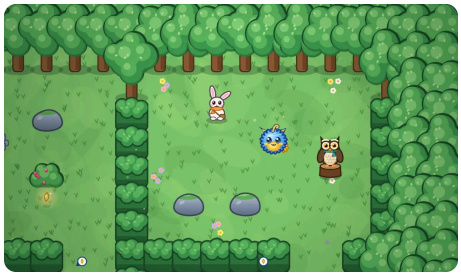
- One hit drops an enemy and three hits drop you, with fast restarts
- Enemy types: pistol thugs, SMG patrols, armored enforcers, shotgunners, bat-wielding bruisers
- A suppressor you toggle with T: quiet but slower rounds, or loud and sharp
- Weapons taken from bodies go into a second slot, switched with 1, 2 or the scroll wheel
- Doors as weapons: slam one into someone to knock them down, then pistol-whip them
- A synthwave soundtrack that adds a lead melody when a fight starts, plus 8-bit sound effects

PROJECT 3 · PROCEDURAL GENERATION AND GUARDED AI DIALOGUE

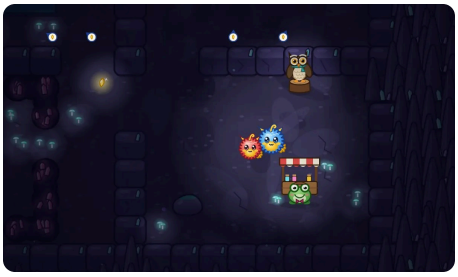
Fuzzle and the Shifting Woods

A top-down roguelite adventure. Fuzzle crosses three cursed lands to gather the Heart Crystals and free his family from the warlock Oink. Every time you fall, the level regrows into a new layout.

Play: shay2222.github.io/fuzzle **Project page**



Mossy Woods



Crystal Caves



Cloud Tops

3
lands, each ending with a rescued family member

100%
of shipped layouts proven solvable by a reachability checker

3
characters with guarded dialogue: hints, a haggling merchant, quests

0
asset files: art in Canvas 2D, music in the Web Audio API



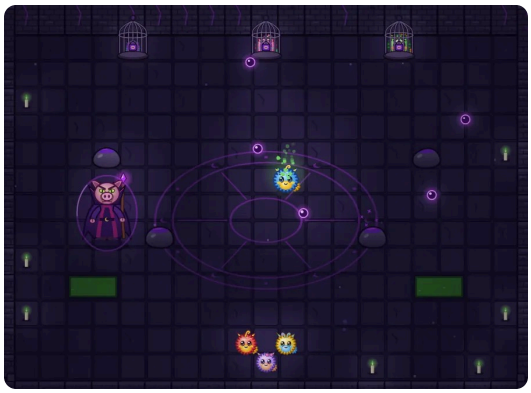
The AI dialogue panel. Mistakes are planted on purpose, such as an invented hazard. The guard flags the draft, it's repaired on retry, and the panel shows each step.

Levels that regrow, but always work

Each layout comes from a seed. It ships only if a checker proves that every seed and the portal can be reached, so procedural variety never strands the player.

An earlier version of the same idea

A generator drafts each line from the real game state. A guard checks it for facts, prices, directions, names and staying in character. A bad draft gets one retry, then a written fallback. Personality settings shift the merchant's tone, but the guard keeps prices inside the game's limits. The Skyward Spell carries this design into C++.

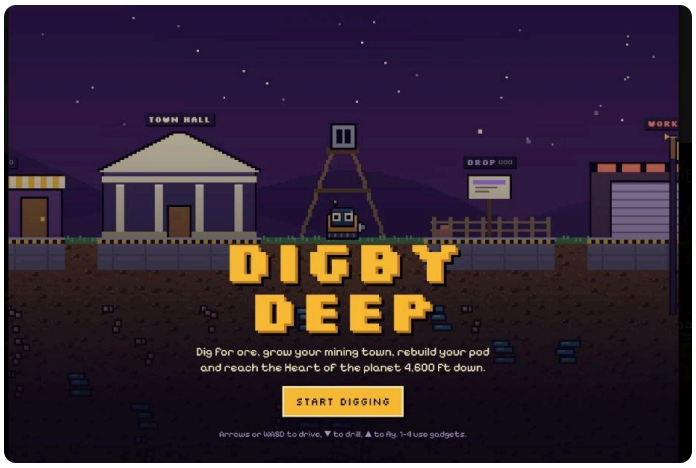


The boss arena. The fight against Oink, with the rescued family caged above.

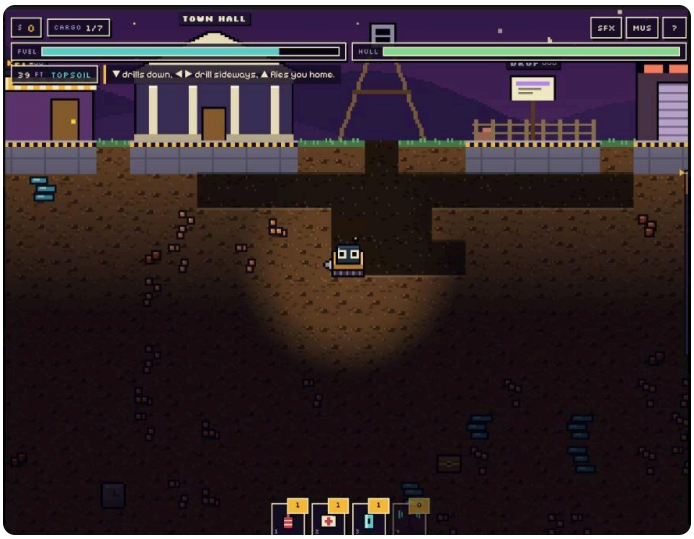
PROJECT 4 · PLAYER CONTROLLER, ECONOMY AND A RESHAPEABLE WORLD

Digby Deep

An 8-bit mining game inspired by Motherload. You drive a small drill pod with animated eyes down through six zones, from Topsoil through the Magma Belt and the Abyss to the Core, 4,600 ft down, where you crack open the planet's Heart.



Title screen. The mining town above the shaft.



Digging. Fuel and hull gauges, cargo, the depth and zone, the gadget bar, and a headlight cut out of the darkness.

4,600 ft
of depth across six zones, ending at the planet's Heart

64 × 376
tile world built from a seed and dug out in real time

9
town buildings, growing the town from Outpost to Capital

~150 KB
in one HTML file, with no images and no game libraries

The loop

Dig ore, fly home before your fuel runs out, sell, upgrade, then dig deeper. Drive, drill and fly all run on one controller.

- Branching pod upgrades: each part splits into one of two specialties, and the top levels cost gems
- Modules and gadgets such as the Ore Scanner, with combinations that unlock bonus synergies
- Contracts with types and rarities, streak bonuses and guild rank
- Changing market prices, hidden relics, and a "new planet" restart after you win

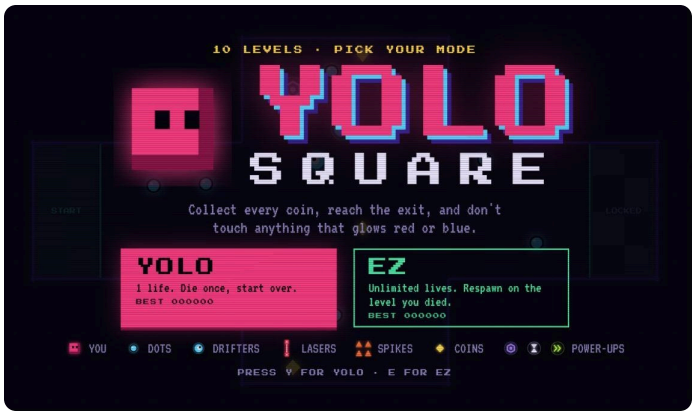
How the code works

- **A seeded world.** The mine is one array of tile types. The same seed always rebuilds the same map, caves are carved by random-walking "worms", and each zone has its own ore odds.
- **Stats from one function.** Saved progress lives in one state object, and one function recalculates every stat from scratch. That removes a whole class of stacking bugs.
- **Fixed-step physics** at 60 Hz with axis-separated collision. Drilling takes time, set by the rock's hardness and the drill's speed.
- **Rule-driven eyes.** They follow the drill, blink, and turn into ^ ^ for ore, X X when hurt, and a frown when the hull is low.
- **Music written as text** and scheduled ahead on the audio clock, with bass lines and arpeggios generated from the chord names.
- **Saves** keep the whole dug-out map, and older saves are converted to the newer format.

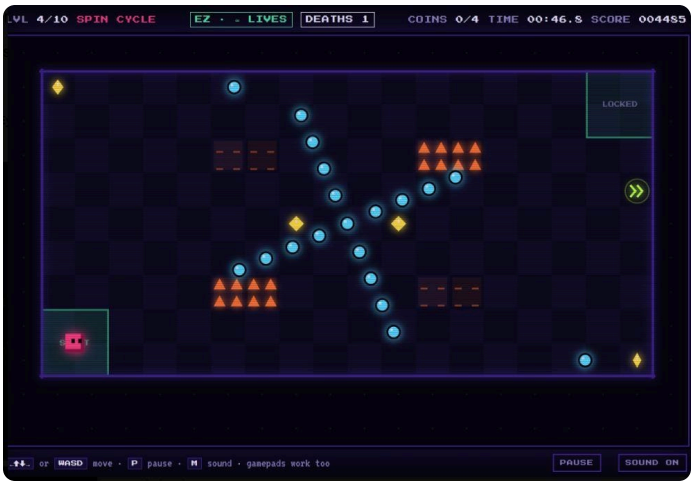
PROJECT 5 · PRECISION ACTION AND DIFFICULTY TUNING

YOLO Square

One square. Ten levels. You only live once. You steer a small pink square with big eyes through rooms full of dodging dots, timed lasers, rising spikes and drifters that move at random. In each level you collect every coin, then reach the exit. The last level is called YOLO.



Pick your mode. The title screen teaches every hazard and power-up at a glance.



Level 4, Spin Cycle. A rotating arm of dots, spike traps, coins and a locked exit, with an EZ-mode death counter.

10
hand-built levels, including a spiral, a staircase, a crossroads and a ring

2
modes for two kinds of players: YOLO and EZ

3
power-ups: shield, slow-mo and speed boost

3
input types: keyboard, gamepad and touch

One choice, two player journeys

YOLO mode gives one life for the whole run, so one hit ends it. EZ mode gives unlimited lives and sends you back to the start of the level you died on, while counting every death. Each mode keeps its own best score, so a careful player and an expert both have something to chase.

Tuning for fairness

Hazard timing and level pacing are tuned so each death reads as the player's mistake. Enemies move at different speeds, and random drifters show with their eyes where they're about to go. That makes randomness readable instead of cheap.

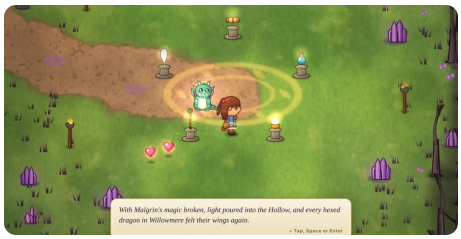
Features

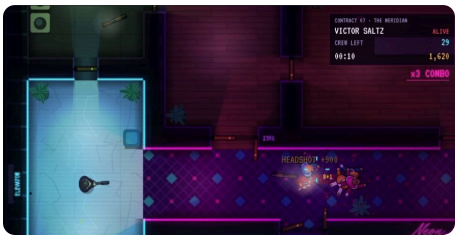
- Collect-every-coin, then reach-the-exit level goals, with steadily rising difficulty
- Dots, timed lasers, rising spikes and random drifters
- Shield, slow-mo and speed-boost power-ups
- Dark neon visuals with a chiptune soundtrack that speeds up each level
- Separate best scores for each mode
- Keyboard, gamepad and touch controls

SUMMARY

Skills shown across the five games

SKILL	WHERE IT SHOWS
C++ systems programming	The Skyward Spell is about 11,300 lines of freestanding C++20 compiled to WebAssembly with no engine and no standard library. It has its own memory arena, containers, maths and text handling.
Rendering and engine work	A signed-distance-field sprite painter and atlas packer, a radix-sorted single-batch renderer, a WebGL 2 ground and lighting shader, and bloom and grading.
Generative AI in gameplay	Prompts built from live game state, JSON replies parsed in C++, and a guard layer of deterministic checks with retry and fallback. The open model runs in the browser on WebGPU, at no cost to players.
Guardrails and logic layers	In both Skyward and Fuzzle, the model or generator proposes and the game decides. Directions, names, items, gifts and spoilers are all checked against game state.
Debugging AI against deterministic logic	An inspector that logs every verdict with its reason, built-in guard tests with known-bad lines, a stand-in model that returns bad replies on purpose, and Fuzzle's planted-mistake mode.
Gameplay and AI behaviour	A companion dragon AI, goblin state machines, and Malgrin's multi-phase fight. In One in the Chamber: six enemy types, door physics as a weapon, and suppressor and ammunition systems.
Procedural generation	Skyward's rule-based lands, with guaranteed crossings and about 6,700 props each. Fuzzle's seeded layouts, which ship only after a solvability check. Digby Deep's seeded mine, with caves carved by random-walking worms.
Game feel and audio	Fixed 120 Hz simulation with interpolation, spring-chain animation, an original C++ synthesiser with eight songs, synthwave combat music, and permanent blood decals.
Tools and testing	Debug exports to jump to any land, place or character, and Python and Playwright tests that play the game from the title screen to the ending.
Player controllers and economy	In Digby Deep, three movement modes run on one controller, the world is reshaped in real time, and the upgrade economy is built on branching specialties, contracts and changing prices.
Difficulty and level design	YOLO Square has ten hand-built levels with tuned hazard timing, and two modes that serve different players from the same content.







Links
The Skyward Spell: shay2222.github.io/skyward-spell · code: github.com/shay2222/skyward-spell · project page: shay2222.github.io/skyward-spell-portfolio
Fuzzle and the Shifting Woods: shay2222.github.io/fuzzle · project page: shay2222.github.io/fuzzle-portfolio